

Use Case: Pre-Commit Secret & PII Sanitizer Agent

Track

Track 6 - AI for Security

Use Case Name

Pre-Commit Secret & PII Sanitizer Agent - Local-First Source Protection

Tagline

Stop sensitive data before it leaves the developer's machine.

Short Version of the Use Case

Developers sometimes accidentally stage API keys, database credentials, private keys, customer data, or production-like personal information in source code, configuration, logs, test fixtures, or infrastructure definitions.

The Pre-Commit Secret & PII Sanitizer Agent runs locally when a developer commits code. It examines only the staged Git diff, detects likely secrets and personally identifiable information (PII), explains the risk, and proposes a safe replacement before any content is pushed to a remote repository.

The solution is deliberately local-first. Deterministic rules and local analysis perform the core detection, so source code and candidate sensitive data never need to leave the developer's machine. **Qwen2.5-Coder 3B Instruct**, run locally, classifies only ambiguous findings; the system remains useful and reliable without an LLM.

Rather than discovering a leak in CI after the code is already shared, the agent prevents the commit at the earliest practical point.

Problem Statement

Accidentally committing sensitive data is a common and costly software-delivery risk.

Developers may unintentionally include:

- API keys, OAuth tokens, passwords, and bearer tokens
- Database connection strings

- Cloud access credentials
- Private keys, certificates, and SSH keys
- Customer email addresses and telephone numbers
- Production data copied into test fixtures or logs
- Secrets embedded in Terraform, YAML, scripts, notebooks, or documentation

Most security scanning happens in continuous integration or after a repository has been pushed. At that point, the sensitive value may already be available to other users, backups, build systems, and repository history.

The organization then has to react:

- Revoke and rotate credentials
- Investigate potential access or exposure
- Remove sensitive content from Git history
- Rebuild and redeploy affected systems
- Notify security, privacy, and compliance stakeholders
- Spend engineering time on urgent remediation

The ideal security control is earlier:

Prevent sensitive data from entering source control before the first commit succeeds.

AI-Native SDLC Alignment

The solution introduces an AI-powered security guardrail directly into the software development lifecycle. Unlike traditional scanners that operate after code is committed, the agent functions as an autonomous SDLC participant: it intercepts commits, evaluates risk, explains findings, proposes remediations, and orchestrates safe code changes before source code enters repository history.

This transforms security validation from a passive, after-the-fact check into an AI-assisted engineering workflow that runs inline with how developers already work:

- **Autonomous risk evaluation** - the policy engine reasons over deterministic signals and, where needed, a local model to reach a decision without human triage for the common case.
- **In-the-loop AI collaboration** - the agent explains *why* a finding is risky and proposes a specific fix, rather than only emitting a pass/fail signal.
- **Human-approved autonomy** - the agent can act (redact, replace, restage) but never merges a fix or completes a commit without developer confirmation.

- **Continuous SDLC embedding** - the same policy and remediation engine can extend from the local hook into CI/CD and platform-level enforcement as the solution matures.

This positions the agent as a native participant in the engineering workflow rather than an optional add-on scanner.

Proposed Solution

Build a **Pre-Commit Secret & PII Sanitizer Agent** that acts as a local security guardrail for developers.

The agent installs a Git pre-commit hook and analyzes staged changes just before the commit completes. It does not scan the whole repository unnecessarily. It looks only at new or changed lines that are about to become part of Git history.

The system provides four levels of protection:

1. Detect - What sensitive data is staged?

The agent checks the staged diff for high-risk patterns, including:

- API keys and access tokens
- Private-key blocks and certificates
- Passwords and connection strings
- JWTs and bearer tokens
- Cloud-provider credential formats
- Email addresses and telephone numbers
- Payment-card-like numbers and customer-record fragments

Example:

Potential AWS credential detected

The staged file contains an access-key pattern next to a secret assignment.

2. Explain - Why is this risky?

The agent provides a clear explanation rather than a vague failed check.

Example:

Commit blocked: likely database credential

config/database.yml contains a connection string with a username and password. If committed, this value can remain available in repository history even after the file is later deleted.

For PII, it can explain the context:

PII warning: customer email found in a test fixture

Test data should use synthetic values. Real or production-like personal data should not enter source control.

The goal is to help the developer understand the exposure and the correct secure alternative.

3. Sanitize - How can I fix it safely?

For each finding, the agent proposes a remediation preview. It never silently changes files.

Possible fixes include:

- Replace a secret with an environment-variable reference
- Replace a secret in an example file with a safe placeholder
- Replace PII with synthetic test data
- Move local configuration to a .env file that is excluded by .gitignore
- Use an approved secret-management reference, such as a Vault or Key Vault URI
- Create a time-bound, reasoned exception for a verified false positive

Example:

Security check failed: commit blocked

File: config/.env.example

Finding: AWS access key

Severity: Critical

Reason: The value matches an access-key format and appears in a credential as signment.

Recommended replacement:

AWS_ACCESS_KEY_ID=\${AWS_ACCESS_KEY_ID}

Actions:

- [R] Preview replacement with a safe placeholder
- [V] Use an environment or Vault reference
- [E] Add a reviewed exception with a reason
- [A] Abort and fix manually

The developer must explicitly choose a remediation before the agent edits and restages a file.

4. Prevent - Can the commit continue?

The policy engine decides the outcome using severity and confidence:

Finding type	Confidence	Action
Private key, clear API key, cloud credential	High	Block the commit
Password in configuration or connection string	High	Block the commit
Likely customer PII	Medium	Warn and require confirmation or replacement
Test fixture or public placeholder	Low	Allow with an auditable exception

This avoids both extremes: it stops obvious, high-impact leaks while limiting friction from low-confidence false positives.

Top 2 Use Cases

Top Use Case 1: Secret Leak Prevention

Prevent API keys, cloud credentials, passwords, JWTs, and connection strings from entering Git history by detecting and replacing them before commit completion.

Top Use Case 2: PII Protection in Development

Prevent customer emails, phone numbers, test data, and production-derived records from being committed into repositories by identifying and sanitizing sensitive content during code commits.

Selected Technology Stack

The MVP uses a local Python stack that can run without administrator rights or a cloud model.

Layer	Selected technology	Purpose
Language	Python 3.11+	Local command-line tool, Git integration, and policy logic
Hook framework	pre-commit	Installs and invokes the Git pre-commit workflow consistently
Secret scanner	detect-secrets	Detects known credential patterns, high-entropy values, and secret assignments
PII scanner	Microsoft Presidio with spaCy	Detects and classifies PII in staged content
Policy and remediation	Custom Python module	Applies confidence rules, previews safe fixes, and records approved exceptions
Terminal experience	rich	Displays readable findings, remediation previews, and confirmation prompts
Optional local model	Qwen2.5-Coder 3B Instruct	Resolves only ambiguous code and configuration findings

Why We Chose `detect-secrets`

Both Gitleaks and `detect-secrets` can detect secrets and integrate with `pre-commit`. The project will use **`detect-secrets`**.

`detect-secrets` is a Python package, so it integrates directly with the custom Python policy engine. It supports the exact pre-commit workflow required by this use case: scan only staged filenames with `detect-secrets-hook`, compare findings against a reviewed `.secrets.baseline`, emit JSON for the policy engine, and add custom filters where organization-specific logic is needed.

This makes it a practical choice for a company laptop and a hackathon MVP:

- No separately downloaded executable is required.

- It is installable with `pip` in a user-level Python environment.
- Its baseline model prevents existing known findings from blocking new work while still stopping newly introduced secrets.
- Its Python API and custom-filter support let the project add its own confidence scoring without parsing terminal text.
- It can run offline; any optional verification behavior that could make a network request is disabled.

Gitleaks remains a strong option for a later CI or enterprise defense-in-depth layer. For this local, Python-first sanitization agent, `detect-secrets` provides the tighter integration path.

Local Qwen Classifier

The selected local model is **Qwen2.5-Coder 3B Instruct**, preferably a quantized GGUF build such as `Q4_K_M`, executed through `Ollama` or `llama-cpp-python`.

Qwen is not the primary secret detector. It is invoked only when deterministic checks produce a medium-confidence finding, for example when the tool needs to distinguish an obvious placeholder from a realistic credential-like fixture. The model receives a redacted candidate and a minimal local code window. It must return a strict classification, confidence score, and short reason without repeating the sensitive value.

Known private keys, recognized access tokens, passwords in connection strings, and other high-confidence secret findings always block the commit without model involvement.

Example classifier contract:

Classify this local security finding as exactly one of:
`REAL_SECRET`, `PII`, `TEST_FIXTURE_OR_PLACEHOLDER`, or `UNKNOWN`.

Do not reproduce the candidate value.

Return JSON: classification, confidence from 0 to 1, and a reason under 20 words.

How the Security Agent Works

The solution uses a layered detection pipeline. Reliable deterministic checks run first; optional AI is used only for ambiguous classification.

Staged Diff Collector

Collects only the files and lines included in `git diff --cached`.

Secret Detection Engine

Uses detect-secrets plugins for known credential patterns, assignment context, and entropy scoring. The hook scans staged filenames only and compares results against a reviewed `.secrets.baseline`.

PII Detection Engine

Detects scoped PII patterns such as email addresses, telephone numbers, and payment-card-like numbers, then evaluates whether they occur in application code, logs, examples, or test fixtures.

Risk and Policy Agent

Combines rule matches, surrounding code context, repository policy, confidence, and severity to decide whether to block, warn, or allow.

Optional Local Verification Agent

Uses Qwen2.5-Coder 3B Instruct locally, for example through Ollama or `llama-cpp-python`, only when deterministic checks cannot distinguish a true secret from a harmless fixture. It receives a redacted candidate with a small local code window and sends nothing to external services.

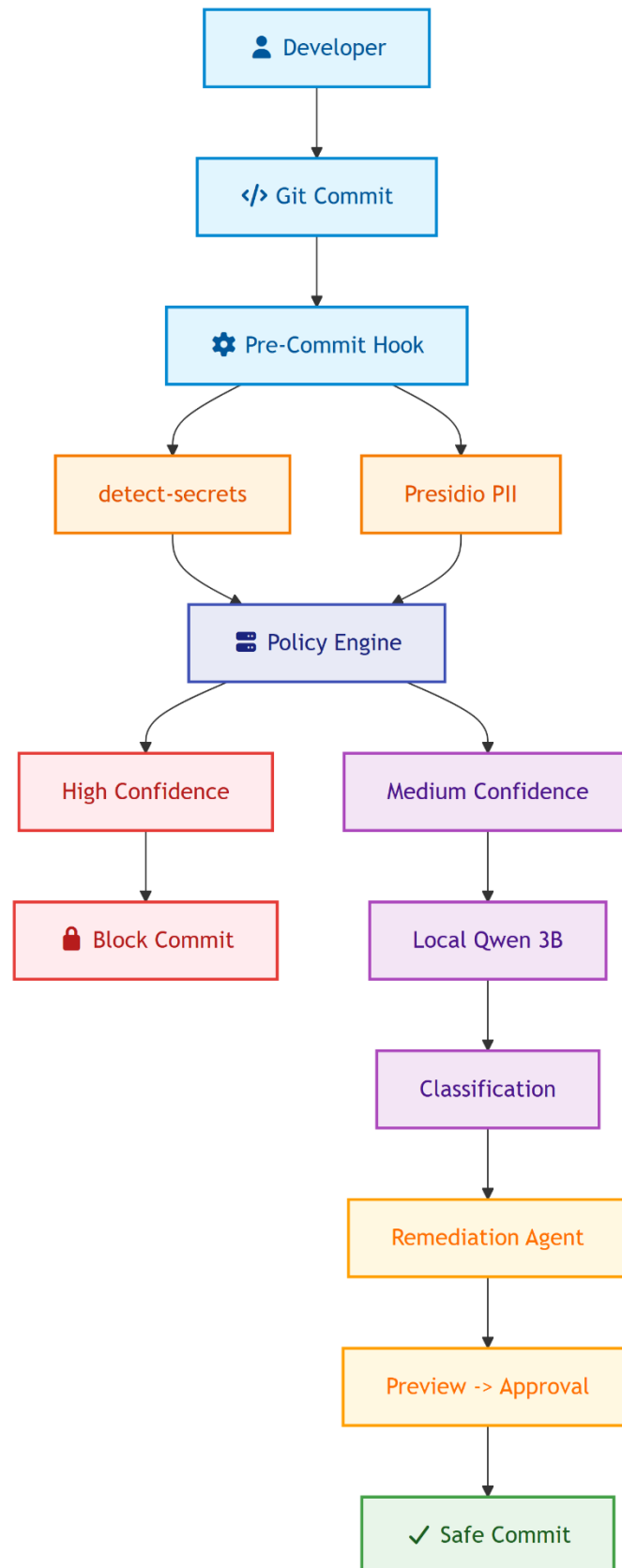
Remediation Agent

Creates a human-readable fix proposal, previews the change, and applies it only after developer approval.

The components work together to answer one central question:

Is this staged change safe to add to source-control history, and what is the safest way to fix it if it is not?

Architecture Diagram



Integrations

The MVP runs entirely as a local Git hook and requires no external integration to function. The architecture is designed so the same policy and remediation engine can extend into an organization's existing engineering ecosystem as the solution matures beyond the hackathon MVP:

- **GitHub Enterprise** - server-side secret-scanning policy alignment and pull-request status checks.
- **Azure DevOps** - pipeline-level enforcement mirroring the local pre-commit policy.
- **GitHub Actions / Azure Pipelines** - CI-stage re-verification as a defense-in-depth layer behind the local hook.
- **Microsoft Defender for DevOps** - centralized security posture reporting across repositories.
- **Azure Key Vault** - approved secret-reference target recommended by the remediation agent instead of a hardcoded value.
- **Jira** - automatic ticket generation for reviewed exceptions that require follow-up remediation or secret rotation.
- **Confluence** - lookup of organization security policy pages to justify why a finding was blocked or allowed.

None of these integrations are required for the hackathon MVP; they define the extension path from a local developer tool to an enterprise-managed control.

Current MVP uses local execution only. Integrations are future-state architecture.

Example End-to-End Scenario

A developer is preparing a feature for commit. The staged changes include:

1. A new `.env.example` file with a copied access key
2. A database URL containing a password
3. A test fixture using a customer's real email address
4. A normal source-code file with no sensitive content

The developer runs:

```
git commit -m "Add notification configuration"
```

The pre-commit agent intercepts the command.

Analysis

Files scanned: 4

High-risk findings: 2

PII findings: 1

Safe files: 1

Why was the commit blocked?

- The environment example includes a credential-looking value.
- The database URL contains a password in clear text.
- The test fixture contains a customer email address.

Recommended changes

Replace the access key and database password with environment-variable references.

Replace the customer email with `customer@example.test`.

The developer approves the remediation preview. The agent updates and restages only the affected lines. The next commit passes successfully.

No secret or PII was pushed, uploaded, or sent to a cloud analysis service.

What Makes This Different?

This is not just another security scanner that reports problems after a push.

It is a local, developer-centered prevention workflow:

Stage -> Detect -> Explain -> Preview -> Confirm -> Sanitize -> Commit safely

The solution differentiates itself through:

Local-First Privacy

Core detection takes place on-device and works without transmitting source code or candidate secrets to a cloud service.

Earliest Possible Intervention

The control executes before the commit is created, rather than waiting for CI or a later security review.

Explainable Findings

Developers see why a value is unsafe and why the proposed alternative is safer.

Human-Controlled Remediation

The agent does not silently rewrite code. It previews a specific change and requires explicit approval.

Focused Scanning

The agent scans only staged changes, making the check fast enough for daily development.

Reliable Detection Design

detect - secrets rules, entropy checks, confidence scoring, and policy thresholds drive the main decision path. Local Qwen classification is optional and never required for blocking a clear secret.

Innovation

The innovation is the combination of:

Shift-Left Security

Protect sensitive content before it becomes part of Git history.

Local Secret and PII Detection

Keep code and sensitive values on the developer's machine during analysis.

Context-Aware Risk Scoring

Evaluate not just a matching pattern, but its surrounding assignment, file type, and confidence level.

Guided Sanitization

Convert a security failure into a safe, actionable remediation workflow.

Developer Learning

Show the developer how to use environment variables, synthetic data, and managed-secret references correctly, improving secure coding behavior over time.

Optional Offline Intelligence

Use Qwen2.5-Coder 3B Instruct locally only for uncertain cases, preserving privacy while reducing false positives.

Responsible AI, Security & Governance

The solution is deterministic-first by design: AI is invoked only for ambiguous, medium-confidence findings and can never be the sole reason a high-risk secret is allowed through. The table below states the specific risks the hackathon template asks for and how each is mitigated.

Risk	Mitigation
Hallucinated classifications	High-risk secrets always use deterministic detection (detect - secrets, Presidio); the local Qwen model only classifies already-ambiguous findings and cannot downgrade a high-confidence block
False positives	Confidence scoring plus mandatory human approval before any remediation is applied or restaged
False negatives	Multiple detection layers (pattern, entropy, keyword, PII/NER) combined with policy rules, rather than a single detection method
Privacy exposure	Zero source-code or secret transmission outside the developer's device; the local model receives only a redacted candidate and a minimal context window
Explainability	Every finding, and every AI classification, must return a stated reason and confidence score rather than a bare verdict
Compliance	Supports secure SDLC, data-protection, and privacy requirements by preventing sensitive data from ever entering source-control history
Misuse	The agent cannot auto-approve or auto-commit a change; it can only block, warn, or propose a preview that requires explicit developer confirmation

Expected Outcome

The solution helps organizations achieve:

- Fewer secrets and personal-data leaks in source control
- Reduced credential-rotation and incident-response effort

- Lower privacy and compliance exposure
- Earlier security feedback for developers
- Reduced false positives through context-aware classification
- More consistent use of environment variables and secret-management systems
- Safer test fixtures and documentation examples
- Zero routine cloud-analysis cost for local pre-commit detection

Business Impact

- Reduce secret exposure incidents by an estimated 80-90%
- Reduce credential-rotation effort by an estimated 70%
- Reduce remediation cost by catching leaks before repository propagation, avoiding history rewrites and emergency rotations
- Improve developer security compliance through inline, explainable feedback at commit time
- Reduce mean time to detect (MTTD) a leaked secret from days to seconds
- Improve audit readiness and governance through recorded, reasoned, time-bound exceptions

The most important outcome is:

Sensitive data is prevented from entering Git history instead of being remediated after exposure.

Hackathon Demonstration

The demonstration uses a small safe demo repository with intentionally fake credentials and synthetic PII.

Step 1 - Show staged changes

Stage four files:

- `.env.example` with a fake AWS-style key
- `config/database.yml` with a demo password in a connection string
- `tests/customer_fixture.json` with a sample customer email and telephone number
- `src/notification.ts` with clean code

Step 2 - Attempt a Git commit

Run:

```
git commit -m "Add notification configuration"
```

Step 3 - Show detection results

The agent reports:

2 high-risk secret findings

1 PII finding

Commit blocked before the repository is updated

Step 4 - Select a remediation

The developer chooses to:

- Replace credentials with environment-variable references
- Replace PII with synthetic test values

Step 5 - Preview and confirm changes

The agent shows the exact diff it will apply. The developer approves it.

Step 6 - Restage and commit safely

The tool restages the corrected files, and the next commit succeeds.

The demonstration proves that the agent catches the leak before any push, with no external service call.

MVP Installation and Hook Configuration

The hackathon build uses the following local installation flow:

```
pip install --user pre-commit detect-secrets presidio-analyzer rich
python -m spacy download en_core_web_sm
detect-secrets scan > .secrets.baseline
pre-commit install
```

The pre-commit configuration runs the project's policy command. That command calls `detect-secrets-hook --baseline .secrets.baseline --json` for staged files, runs Presidio on the same staged content, and then applies the block, warn, or allow policy. Network verification is disabled so the hook remains local.

Demo Readiness & Roadmap

Current Demo Readiness

Working

- Git pre-commit hook
- Staged diff analysis
- detect-secrets integration
- Presidio PII detection
- Policy engine
- Commit blocking
- Remediation preview
- Safe file restaging

Partially Complete

- Local Qwen classification
- Confidence tuning
- Exception approval workflow

Not Yet Complete

- Enterprise dashboard
- DevOps integrations
- Organization-wide policy management

30-Day Roadmap

- Stabilize the local agent
- Expand secret detection coverage
- Improve remediation suggestions
- Reduce false positives

60-Day Roadmap

- GitHub integration
- Azure DevOps integration
- Centralized policy management
- Team reporting dashboard

90-Day Roadmap

- Enterprise rollout
- Key Vault integration

- Security analytics dashboard
- Organization-wide governance controls
- Adaptive AI policy learning

Success Metrics

The solution can measure:

Detection

Percentage of planted high-confidence credentials blocked before commit

Privacy

Sensitive source content transmitted to external services: zero

Speed

Pre-commit scan duration for a normal staged diff

Accuracy

False-positive rate for fixtures, placeholders, and public examples

Remediation

Percentage of findings resolved through an approved safe replacement

Prevention

Number of secrets and PII findings stopped before entering Git history

A defensible hackathon target is:

100% of planted high-confidence credentials blocked
0 sensitive values transmitted off-device
Under 2 seconds for a normal staged diff
Successful remediation and commit for all demo findings

Final Value Proposition

Teams often detect credentials and personal data only after source code has already been shared, built, or stored in repository history. The resulting cleanup is disruptive, expensive, and difficult to guarantee.

The Pre-Commit Secret & PII Sanitizer Agent moves security to the moment where prevention is simplest: the developer's local commit.

Instead of merely reporting a leak, it explains the exposure, proposes a secure alternative, and keeps the developer in control of the correction.

“Security should stop a secret before the first commit, not report it after it is already shared.”

This creates a practical security habit: every commit is an opportunity to keep credentials, customer data, and source-control history safe.